Software Engineering Lab

Lab 34

Chaos Engineering & Fault Injection

Membuktikan Ketahanan Sistem Terdistribusi Secara Empiris

Lukman / lukman-ss

Ketahanan Sistem Tanpa Bukti Empiris Adalah Asumsi

Layanan terdistribusi rentan ambruk saat downstream mengalami latensi atau outage mendadak

MASALAH UTAMA

Klaim ketahanan arsitektur microservice sering kali hanya diuji pada happy path dan mock unit test sederhana.
Saat ketergantungan downstream melambat atau gagal, ketiadaan perlindungan memicu antrean thread dan resource exhaustion.
Tanpa pengujian empiris terencana, insiden production pertama menjadi tempat uji coba yang mahal bagi reputasi.

BAHAYA CASCADING

Latensi downstream 500ms dapat mengunci connection pool HTTP client dalam hitungan detik.
Kegagalan satu payment gateway merembet ke checkout, cart, hingga mematikan API gateway hulu.
Pengujian integrasi statis tidak mampu mensimulasikan kegagalan jaringan yang fluktuatif.

Membangun framework eksperimen chaos deterministik di Go 1.22+ murni tanpa external library.

Menyuntikkan latensi dan error downstream secara terprogram di memori.

Membuktikan isolasi kegagalan lewat Circuit Breaker dan graceful degradation fallback.

Resilience without empirical verification is merely wishful thinking.

Empat Langkah Baku Eksperimen Ilmiah Perangkat Lunak

Chaos engineering merupakan disiplin verifikasi terstruktur, bukan aksi merusak server secara acak

1. STEADY STATE

Definisikan metrik kesehatan output yang dirasakan pengguna, seperti rasio error dan throughput transaksi.
Fokus pada metrik eksternal bisnis, bukan metrik internal sistem seperti beban CPU atau penggunaan RAM node.
Tentukan baseline angka normal sebelum eksperimen dijalankan sebagai acuan toleransi.

2. HIPOTESIS & INJEKSI

Rumuskan hipotesis: 'Steady-state error rate tetap di bawah ambang batas meskipun downstream mati'.
Suntikkan gangguan terkontrol: latensi jaringan tiruan atau pemaksaan error status downstream.
Jalankan injeksi pada durasi terukur dengan interval observasi metrik periodik.

Tetapkan safety guardrails ketat: auto-abort langsung aktif saat metrik melanggar threshold toleransi.

Sinkronisasi pembersihan fault injector seketika untuk mengembalikan trafik normal.

Buktikan bahwa sistem memulihkan diri secara mandiri tanpa intervensi manual tim engineer.

Define steady state first, inject controlled turbulence, and abort automatically on breach.

Steady State: Mengukur Dampak Pengguna, Bukan Node

Kesehatan arsitektur diukur dari error rate transaksi akhir, bukan persentase utilitas hardware

METRIK OUTPUT VS INTERNAL

Metrik internal (CPU 80%, RAM 4GB) sering menipu: node sibuk belum tentu melayani pengguna dengan benar.
Metrik output (Error Rate, Order Completion Rate) langsung mencerminkan kepuasan konsumen.
Lab menetapkan ambang batas error rate maksimum (contoh: 20% - 25%) sebagai batas steady state.

SAMPLE WARMUP MINIMUM

Evaluasi kesehatan membutuhkan minimal 5 sampel request awal sebelum perhitungan error rate dieksekusi.
Mencegah false alarm saat request pertama kebetulan gagal pada saat inisialisasi koneksi.
Memastikan rasio error dihitung berdasarkan data agregat yang cukup representatif.

TotalRequests: akumulasi seluruh request masuk yang dilayani client.

FailedRequests: jumlah request yang berujung kegagalan atau error.

SuccessRequests: request yang berhasil diselesaikan via jalur utama maupun fallback.

Measure customer-facing output metrics to define health, never component internals.

Topologi Komponen dan Siklus Closed-Loop Kontrol

Pemisahan tegas antara injeksi kegagalan, mitigasi client, monitor metrik, dan abort runner

DIAGRAM ALUR SISTEM

```
[Client Request] --> [Resilient Client (Circuit Breaker)]
|--> Downstream Service (with Injector)
|--> Graceful Fallback (on Failure / Open)
[Steady State Monitor] <-- (Records Success / Failure)
[Experiment Runner] checks Monitor Ticker --> Triggers Auto-Abort
```

internal/fault: Mengatur latensi buatan dan pemaksaan error downstream secara thread-safe.

internal/circuitbreaker: State machine 3 fasa (Closed, Open, Half-Open) pengendali akses.

internal/monitor: Pelacak metrik rasio kegagalan berbasis operasi atomik berkecepatan tinggi.

internal/experiment: Orkestrator siklus hidup pengujian dengan kemampuan auto-abort sinkron.
Runner tidak berjalan buta; runner membaca kondisi monitor secara periodik via background ticker.

Jika monitor melaporkan status tidak sehat, runner seketika menetralkan injector downstream.

Mengeliminasi risiko eksperimen tertinggal yang merusak trafik production secara permanen.

A closed-loop monitor guarantees turbulence stops before customer harm becomes permanent.

Injeksi Fault: Simulasi Latensi dan Error Terprogram

Implementasi thread-safe menggunakan sync.RWMutex dan time.After dengan pembatalan konteks

```
func (fi *Injector) Execute(ctx context.Context) error {
    fi.mu.RLock()
    enabled, latency, forceErr := fi.enabled, fi.latency, fi.forceError
    fi.mu.RUnlock()
    if !enabled { return nil }
    if latency > 0 {
        select {
        case <-time.After(latency):
        case <-ctx.Done(): return ctx.Err()
        }
    }
    if forceErr { return ErrInjectedFault }
    return nil
}
```

Latensi injeksi mendengarkan ctx.Done() agar request timeout client langsung dibatalkan.

Mencegah goroutine leak saat client membatalkan transaksi di tengah penundaan waktu.

Mendukung timeout standar library context Go secara alami tanpa blocking abadi. Operasi SetFault() dan Clear() menggunakan Lock eksklusif untuk mutasi state instan.

Operasi Execute() dan IsEnabled() menggunakan RLock untuk throughput pembacaan tinggi.

Terbukti bebas race condition di bawah pengujian go test -race.

Inject faults at system boundaries with context awareness to prevent goroutine leaks.

State Machine Circuit Breaker: Tiga Fasa Proteksi

Mendeteksi kegagalan beruntun, memutus arus panggilan, dan menguji pemulihan otomatis

CLOSED STATE (NORMAL)

Permintaan dialirkan langsung ke downstream service tanpa hambatan.
Setiap kegagalan menaikkan counter failures secara kumulatif.
Jika counter failures mencapai threshold, state seketika melompat ke OPEN.

OPEN STATE (ISOLASI)

Permintaan langsung dialihkan ke fallback tanpa pernah memanggil downstream (fail-fast).
Melindungi downstream yang sedang sekarat dari gempuran beban trafik baru.
Setelah masa cooldown terlewati, pemanggilan berikutnya menggeser state ke HALF-OPEN.

Mengizinkan satu panggilan uji coba menerobos ke downstream untuk mengetes kesehatan.

Jika panggilan uji coba berhasil: state kembali ke CLOSED dan failure counter di-reset nol.

Jika panggilan uji coba gagal: state langsung terhempas kembali ke OPEN dengan cooldown baru.

A circuit breaker isolates downstream failures and probes recovery without manual restarts.

Eksekusi Terproteksi dan Pengalihan Fallback

Struktur eksekusi fungsi membungkus logika utama dengan pemulihan otomatis dan isolasi error

```
func (cb *CircuitBreaker) Execute(fn, fallback func() error) error {
    cb.mu.Lock()
    cb.checkStateLocked()
    if cb.state == StateOpen {
        cb.mu.Unlock()
        if fallback != nil { return fallback() }
        return ErrCircuitOpen
    }
    cb.mu.Unlock()
    err := fn()
    // Update state & trigger fallback on error
    return cb.handleResult(err, fallback)
}
```

Mutex dilepas sebelum eksekusi fn() untuk mencegah contention saat downstream lambat.

Pengecekan cooldown dilakukan di checkStateLocked() sebelum status OPEN dievaluasi.

State transitions terlindungi penuh tanpa mengunci thread eksekusi I/O utama. Jika fallback didefinisikan, error downstream ditelan dan fallback dieksekusi transparan.

Jika fallback bernilai nil, error asli atau ErrCircuitOpen diteruskan ke pemanggil.

Memungkinkan fleksibilitas tinggi antara fail-silent atau fail-explicit.

Release locks before downstream calls to avoid thread lockups during degraded latency.

Graceful Degradation: Menyelamatkan Metrik Bisnis

Ketika primary gateway gagal, antrean fallback memastikan pesanan tidak hilang sia-sia

JALUR PRIMARY VS FALLBACK

Jalur Utama: Pemanggilan sinkron langsung ke Payment Gateway eksternal.
Jalur Fallback: Memasukkan payload transaksi ke antrean asinkron (Payment Queued).
Pengguna menerima konfirmasi antrean alih-alih layar error HTTP 500 yang merusak konversi.

DAMPAK PADA METRIK STEADY-STATE

Monitor mencatat request fallback sebagai SUCCESS di sisi pengalaman konsumen.
Tingkat kegagalan sistem tetap terjaga pada 0.00% meskipun downstream mengalami pemadaman 100%.
Circuit Breaker mencegah lonjakan latensi dengan mengalihkan request secara instan (<1ms).

Fallback harus beroperasi lokal atau menggunakan infrastruktur penyimpanan yang terisolasi.

Jangan menggunakan fallback yang bergantung pada dependensi jaringan yang sama dengan primary.

Pastikan ada worker rekonsiliasi asinkron untuk memproses ulang data antrean saat downstream pulih.

Graceful fallbacks convert hard outages into delayed asynchronous completions.

Pelacakan Rasio Kegagalan Menggunakan Operasi Atomik

Mengeliminasi overhead penguncian pada jalur transaksi dengan memanfaatkan paket sync/atomic

```
func (m *Monitor) RecordSuccess() {
    atomic.AddUint64(&m.totalRequests, 1)
}
func (m *Monitor) RecordFailure() {
    atomic.AddUint64(&m.totalRequests, 1)
    atomic.AddUint64(&m.failedRequests, 1)
}
func (m *Monitor) IsHealthy() bool {
    total := atomic.LoadUint64(&m.totalRequests)
    if total < 5 { return true } // Warmup minimum
    return m.ErrorRate() <= m.maxErrorRate
}
```

atomic.AddUint64 berjalan pada level instruksi CPU (lock-free) tanpa blokade mutex antrean.

Cocok untuk throughput jutaan request per detik di lingkungan microservice berskala besar.

Menjamin akurasi penghitungan total dan kegagalan pada pengujian multi-goroutine. ErrorRate() menghitung rasio gagal terhadap total secara real-time.

Threshold kesehatan maxErrorRate (misal 0.20 atau 20%) menjadi dasar evaluasi IsHealthy().

Kombinasi ambang batas dan minimum sample mencegah false positive selama fase rintisan.

Lock-free atomic counters track health metrics without adding contention to hot paths.

Orkestrator Eksperimen dan Mekanisme Auto-Abort

Loop monitoring periodik menjamin penghentian darurat seketika saat metrik terdegradasi

```
func (e *Experiment) Run(ctx context.Context) error {
    e.injector.SetFault(e.config.Latency, e.config.ForceError)
    ticker := time.NewTicker(e.config.MonitorInterval)
    defer ticker.Stop()
    for {
        select {
        case <-ctx.Done(): e.terminate(StateAborted, ...)
        case <-timeout:   e.terminate(StateCompleted, ...)
        case <-ticker.C:
            if !e.monitor.IsHealthy() {
                e.terminate(StateAborted, reason)
                return fmt.Errorf("aborted: %s", reason)
            }
        }
    }
}
```

Fungsi terminate() memanggil e.injector.Clear() secara langsung di dalam blok mutasi state.

Menjamin injeksi latensi dan error mati dalam hitungan milidetik saat ambang batas terlanggar.

Mencegah efek samping tertinggal yang membahayakan siklus pengujian berikutnya. Runner menyimpan string alasan pembatalan (abortReason) secara transparan untuk audit log.

Menyajikan persentase pasti pelanggaran error rate pada saat eksperimen dihentikan paksa.

Mempermudah identifikasi kelemahan arsitektur sebelum melanjutkan ke perbaikan berikutnya.

An automated kill switch is mandatory before injecting turbulence into any system.

Pengendalian Blast Radius: Mencegah Malapetaka

Tiga pilar proteksi untuk memastikan pengujian chaos tidak menimbulkan kerugian nyata

BATASAN TEMPORAL (WAKTU)

Setiap eksperimen wajib memiliki durasi waktu maksimum yang kaku (misal 300ms - 500ms).
Jika waktu habis sebelum terjadi anomali, eksperimen selesai dengan status COMPLETED.
Tidak boleh ada eksperimen chaos yang dibiarkan aktif tanpa batas waktu akhir.

BATASAN METRIK (HEALTH)

Pemeriksaan kesehatan dilakukan berulang pada interval pendek (misal 20ms - 40ms).
Pelanggaran ambang batas langsung memicu auto-abort tanpa menunggu durasi eksperimen selesai.
Menahan dampak kegagalan agar tidak pernah menyebar ke lebih dari persentase toleransi.

Metode Clear() mengembalikan flag enabled ke false dan mereset nilai latensi menjadi nol.

Pembersihan dieksekusi sinkron di terminate(), bukan mengandalkan garbage collector atau cron.

Sistem kembali ke status baseline sejati dalam durasi kurang dari 1 milidetik.

Timeouts, metric thresholds, and immediate cleanups define the blast radius sandbox.

Demo Kasus Pembayaran: Baseline dan Mitigasi Sukses

Simulasi interaktif membuktikan kestabilan baseline dan keampuhan graceful fallback

FASE 1: BASELINE (REQ #1 - #5)

5 request dikirim dalam kondisi downstream normal tanpa adanya gangguan buatan.
Seluruh request berhasil dilayani lewat jalur utama: 'Payment Success (Primary)'.
Circuit Breaker bertahan di state CLOSED; error rate monitor tercatat 0.00%.

FASE 2: RESILIENT CLIENT (REQ #6 - #15)

Eksperimen menyuntikkan kegagalan error 100% pada downstream service selama 300ms.
Req #6 & #7: Gagal di primary, dialihkan mulus ke 'Payment Queued (Fallback)' (CB: CLOSED).
Req #8: Threshold 3 kegagalan terlampaui; Circuit Breaker seketika meloncat ke OPEN.
Req #9 - #15: Fast-fail tanpa memanggil downstream, langsung mengeksekusi fallback (CB: OPEN).

Total Requests: 15 | Success: 15 | Failed: 0 | Error Rate: 0.00%.

Eksperimen selesai dengan status COMPLETED karena steady-state kesehatan tidak pernah terlanggar.

Pelanggan tetap terlayani tanpa menyadari adanya pemadaman total di downstream gateway.

Graceful degradation keeps error rate at zero even while the circuit breaker trips open.

Demo Fase 3 & 4: Auto-Abort dan Pemulihan Mandiri

Eksperimen tanpa mitigasi memicu pembatalan otomatis, diikuti pemulihan penuh trafik normal

FASE 3: AUTO-ABORT UNMITIGATED

Menjalankan eksperimen kedua pada raw service call tanpa perlindungan fallback.
Raw Call #1 & #2 gagal berturut-turut, mendongkrak error rate melompat ke 33.33%.
Ambang batas 20% dilanggar: Runner langsung membatalkan pengujian dengan status ABORTED.
Alasan pembatalan: 'Steady state breached: error rate 33.33% exceeded threshold'.
Fault Injector langsung dinetralkan otomatis (Active: false).

FASE 4: TRAFIK RECOVERY (REQ #16 - #20)

Setelah jeda cooldown 200ms, trafik produksi normal dialirkan kembali ke sistem.
Req #16 melintas pada mode HALF-OPEN, downstream sukses merespons, state reset ke CLOSED.
Req #17 - #20 meluncur mulus via jalur utama 'Payment Success (Primary)' (CB: CLOSED).

Sistem berhasil melindungi diri sendiri dari gangguan yang melampaui batas toleransi.

Pemulihan berlangsung sepenuhnya otomatis tanpa restart service atau intervensi operator.

Unmitigated turbulence triggers instant auto-abort, enabling self-healing recovery.

Verifikasi Pengujian: Lima Test Case Lolos 100%

Rangkaian unit test dan race detector membuktikan akurasi state dan thread-safety

Test Suite	Target Verifikasi	Hasil
TestFaultInjector	Latensi, Force Error, & Pembersihan Clear()	PASS (0.01s)
TestCircuitBreakerStateTransitions	Closed -> Open -> Half-Open -> Closed	PASS (0.06s)
TestCircuitBreakerGracefulDegradation	Eksekusi fallback menelan error downstream	PASS (0.00s)
TestExperimentAutoAbortOnSteadyStateViolation	Auto-abort saat steady-state terlanggar	PASS (0.01s)
TestConcurrencyAndRace	50 Goroutine konkuren membaca &	PASS (0.00s)

Perintah eksekusi: go test -race ./...

Output: ok labs/34-chaos-engineering/tests 1.481s.

Nol data race terdeteksi pada seluruh interaksi mutex dan atomic pointer.
Seluruh test dirancang deterministik tanpa ketergantungan pada port jaringan nyata.

Menjamin build pipeline CI/CD dapat mengeksekusi suite chaos dalam waktu di bawah 2 detik.

Automated test assertions with race detection turn theoretical resilience into verified fact.

Desain Konkurensi: Harmonisasi RWMutex dan Mutex

Pemisahan strategi sinkronisasi berdasarkan pola akses read-heavy dan write-heavy

RWMUTEX PADA FAULT INJECTOR

Pola pembacaan (Execute, IsEnabled) sangat dominan selama ribuan request per detik.
sync.RWMutex memungkinkan banyak goroutine membaca konfigurasi fault secara simultan.
Write lock hanya diambil sesaat saat injeksi diaktifkan (SetFault) atau dinetralkan (Clear).

MUTEX PADA CIRCUIT BREAKER

Setiap transisi state melibatkan pembaruan variabel kegagalan dan timestamp lastStateChg.
sync.Mutex standar menjamin serialisasi transisi state yang presisi tanpa race condition.
Lock dilepas sebelum pemanggilan delegat fn() untuk mencegah thread bottleneck.

Penghitungan metrik tidak menggunakan lock sama sekali pada level agregasi.

atomic.AddUint64 dan LoadUint64 memastikan pembacaan throughput zero-latency.

Menghilangkan risiko deadlock antara monitor dan komponen pemanggil transaksi.

Match the concurrency primitive to the access pattern: atomic for counters, RWMutex for flags.

Perbandingan: Sistem Naif vs Sistem Berketahanan

Perbedaan perilaku sistem saat menghadapi kegagalan downstream pada skala produksi

Dimensi Evaluasi	Sistem Naif (Tanpa Chaos/CB)	Resilient Client (Lab 34)
Downstream Latency	Thread terkunci, connection pool habis	Fast-fail lewat Circuit Breaker (<1ms)
Downstream Outage	HTTP 500 error langsung ke konsumen	Graceful fallback (antrean transaksi)
Error Rate Pengguna	Melonjak drastis hingga 100%	Tetap 0.00% berkat fallback terisolasi
Verifikasi Ketahanan	Tebakan pasif / manual staging	Eksperimen terprogram & auto-abort
Pemulihan Layanan	Membutuhkan restart service manual	Otomatis pulih via probe Half-Open

Sistem naif membiarkan kegagalan downstream merembet ke seluruh ekosistem layanan.

Sistem berketahanan mengisolasi anomali pada batas perimeter dan meredam dampaknya.
Mencegah kerugian finansial akibat transaksi belanja yang gagal terproses.

Menjaga service level objective (SLO) ketersediaan tetap prima di hadapan pelanggan.

Resilient systems bend under turbulence through isolation; naive systems shatter entirely.

Trade-off dan Batasan Implementasi Laboratorium

Penyederhanaan model lab untuk kejelasan konsep versus kebutuhan sistem production nyata

PENYEDERHANAAN LAB

Metrik Kumulatif: Lab menggunakan counter kumulatif sederhana; production membutuhkan sliding window.
Injeksi In-Memory: Simulasi dilakukan dalam runtime Go lokal, bukan manipulasi paket jaringan (iptables/eBPF).
Error Rate Sederhana: Belum memperhitungkan latensi persentil p95/p99 atau anomali jitter.

FIELD UNUSED DI INJECTOR

Field errorRate float64 pada internal/fault/injector.go tidak diaktifkan pada logika eksekusi.
Lab sengaja menggunakan determinisme penuh via forceError bool untuk menjamin hasil test yang stabil.
Hindari klaim pengujian probabilistik parsial pada implementasi lab saat ini.

Penerapan circuit breaker menambahkan beberapa nanodetik latensi CPU per transaksi.

Penyimpanan fallback antrean membutuhkan alokasi storage dan worker pemrosesan asinkron.

Investasi arsitektur sebanding dengan proteksi downtime yang diperoleh di production.

Acknowledge lab simplifications honestly: cumulative counters teach the logic, sliding windows run production.

Jalur Evolusi Menuju Skala Produksi Enterprise

Mentransformasikan implementasi lab menjadi platform chaos engineering tingkat lanjut

SLIDING TIME WINDOWS

Gantikan counter seumur hidup dengan ring-buffer 60 detik (misal Hystrix / Resilience4j style).
Mencegah riwayat kegagalan masa lalu membebani keputusan isolasi pada trafik masa kini.
Menghasilkan evaluasi kesehatan real-time yang jauh lebih responsif terhadap perubahan kondisi.

EBPF & SERVICE MESH INJECTION

Pindahkan injeksi dari kode aplikasi internal ke layer infrastruktur (Envoy proxy / Chaos Mesh).
Menyuntikkan packet loss, TCP drop, dan DNS failure tanpa mengubah satu baris pun kode bisnis.
Menguji ketahanan sistem secara transparan pada level kernel sistem operasi.

Batasi injeksi hanya pada 1% trafik pengguna nyata melalui header routing khusus (canary).

Meminimalkan dampak bisnis jika terjadi unexpected cascading failure saat pengujian berlangsung.

Integrasi dengan OpenTelemetry distributed tracing untuk memvisualisasikan jalur fault.

Progress from in-memory mocks to eBPF mesh injection as chaos maturity grows.

Jebakan Umum dalam Mengadopsi Chaos Engineering

Kesalahan fatal yang dapat mengubah latihan ketahanan menjadi bencana downtime nyata

1. MERUSAK TANPA AUTO-ABORT

Menyuntikkan kegagalan tanpa mekanisme monitoring otomatis dan tombol pemutus darurat.
Ketika sistem benar-benar ambruk, engineer panik mencari cara mematikan skrip pengujian.
Jangan pernah mengeksekusi eksperimen tanpa loop kill-switch yang terverifikasi.

2. MENGUKUR METRIK YANG SALAH

Mengukur utilitas CPU, memori, atau IOPS server alih-alih keberhasilan transaksi bisnis.
CPU rendah saat server down sering disalahartikan sebagai kondisi sistem yang 'santai'.
Selalu jadikan kepuasan pengguna akhir dan error rate transaksi sebagai kompas utama.

Menerapkan Circuit Breaker tetapi membiarkan error ErrCircuitOpen meledak ke pengguna akhir.

Circuit Breaker hanya memotong beban downstream; tanpa fallback, pengguna tetap menerima error.

Kombinasikan selalu circuit breaker dengan strategi graceful degradation yang matang.

Never inject chaos without an automated kill switch and a customer-preserving fallback.

Panduan Keputusan: Kapan Menggunakan Chaos Engineering

Kriteria kematangan arsitektur sebelum memulai eksperimen gangguan terprogram

SAAT TEPAT MENGGUNAKAN

Arsitektur Microservices dengan ketergantungan downstream kritis (Payment, Logistik, Auth).
Sistem yang memiliki Service Level Agreement (SLA) 99.9% ke atas dengan beban transaksi masif.
Ketika tim telah mengimplementasikan timeout, retry, dan circuit breaker dan ingin bukti ketahanan.
Sebelum menghadapi lonjakan trafik musiman (Flash Sale, Black Friday, promo puncak).

SAAT HARUS MENGHINDARI

Aplikasi monolitik sederhana tanpa ketergantungan jaringan eksternal.
Sistem yang belum memiliki observabilitas dasar (metrik error rate dan logging belum tersedia).
Ketika tim belum menerapkan penanganan error dan timeout standar pada client HTTP/gRPC.
Menginjeksi chaos pada sistem yang sudah diketahui rapuh hanya membuang waktu tanpa nilai tambah.

Observabilitas matang, arsitektur fallback terpasang, dan safety guardrails teruji 100%.

Fix fundamental error handling before adopting chaos; test resilience only after building it.

Checklist Kesiapan Pengujian Chaos di Lingkungan Nyata

Daftar periksa esensial sebelum menyuntikkan gangguan terkontrol ke sistem produksi

FASE PERSIAPAN (PRE-FLIGHT)

[x] Definisikan steady-state berbasis metrik bisnis pengguna (maksimal error rate, latensi p95).
[x] Tentukan hipotesis eksperimen yang spesifik dan terukur.
[x] Konfigurasikan durasi eksperimen dan ambang batas toleransi blast radius.

FASE EKSEKUSI (IN-FLIGHT)

[x] Pastikan background ticker monitor aktif membaca metrik kesehatan secara kontinu.
[x] Verifikasi bahwa fallback menangani kegagalan downstream tanpa mencemari pengalaman pengguna.
[x] Uji fungsi auto-abort darurat untuk memastikan pembatalan otomatis berfungsi saat threshold breached.

[x] Verifikasi metode Clear() menetralkan seluruh gangguan seketika.

[x] Pantau pemulihan otomatis Circuit Breaker dari Half-Open kembali ke Closed.

[x] Dokumentasikan temuan eksperimen ke dalam engineering audit record untuk perbaikan sistem.

Run through the pre-flight checklist: measure steady state, automate abort, verify clean reset.

Rangkuman Esensial: Ketahanan Terverifikasi Secara Empiris

Tiga pilar fundamental dalam membangun sistem yang tangguh menghadapi turbulensi

1. EKSPERIMEN BUKAN KERUSAKAN

Chaos engineering adalah sains verifikasi terencana dengan hipotesis dan batas toleransi.
Tujuannya adalah menemukan kelemahan arsitektur sebelum insiden nyata merugikan pengguna.
Steady state yang sehat berpusat pada output akhir, bukan kondisi internal hardware.

2. ISOLASI & DEGRADASI

Circuit Breaker bertindak sebagai pemutus sekring untuk mencegah kegagalan downstream meluas.
Graceful fallback mengubah error fatal menjadi proses tunda yang menyelamatkan transaksi bisnis.
Kombinasi keduanya menjaga metrik kepuasan pengguna tetap sempurna di tengah bencana sistem.

Jangan pernah menjalankan pengujian chaos tanpa loop monitor otomatis dan safety kill-switch.

Netralisasi fault secara sinkron menjamin kembalinya stabilitas sistem tanpa efek samping.

Verifikasi konkurensi dengan race detector untuk menjamin keandalan pada trafik tinggi.

Resilience is proven only when graceful degradation absorbs faults and safety switches halt chaos.

Eksperimen terencana, interpretasi yang teliti.

Continuous resilience verification separates robust engineering from naive assumption.

Software Engineering Lab 34 – Chaos Engineering & Fault Injection

Software Engineering Lab: 34 - Chaos Engineering & Fault Injection
Implementasi: internal/fault, internal/circuitbreaker, internal/monitor, internal/experiment
Repository: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/34-chaos-engineering
Commit Snapshot: b41fffba

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/34-chaos-engineering`

Lukman / lukman-ss

Software Engineering Lab: 34 - Chaos Engineering & Fault Injection
Implementasi: internal/fault, internal/circuitbreaker, internal/monitor, internal/experiment
Repository: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/34-chaos-engineering
Commit Snapshot: b41fffba
Penulis: Lukman / lukman-ss
Status Pengujian: PASS (5/5 unit & race tests lolos)
Race Detection: PASS (go test -race ./... clean)
Demo Output: Baseline OK, Resilient Fallback OK, Auto-Abort OK, Recovery OK
Lab menggunakan counter kumulatif in-memory untuk determinisme pengujian.
Di lingkungan produksi, terapkan sliding time window dan mesh network injection.